DIAGRAMA DE ARQUITECTURA Y COMPONENTES DE LOS SISTEMAS WEB EN DESARROLLO, DETALLANDO LA ESTRUCTURA LÓGICA Y TECNOLÓGICA, INCLUYENDO MÓDULOS DE ANGULAR, LIBRERÍAS COMPLEMENTARIAS, SERVICIOS API, CONEXIONES GRAPHQL Y OTROS FRAMEWORKS

1) Propósito del entregable

Este informe documenta el análisis de arquitectura Frontend de sistemas web orientados al Sector Educativo y presenta el diagrama de arquitectura y componentes solicitado: estructura lógica/tecnológica, módulos de Angular, librerías complementarias, servicios API, conexiones GraphQL y frameworks involucrados.

En el mes de enero, el comportamiento funcional se sostuvo principalmente con localStorage y servicios mock; pero también se consolidó la preparación e integración GraphQL para autenticación y carga.

2) Resumen ejecutivo

Principios Enero (foco frontend + simulación local)

- Consolidación del frontend Angular y flujo de carga masiva.

- Validación de plantillas Excel en cliente.

- Generación y control de credenciales locales.

- Persistencia con localStorage (archivos, sesión, estado de credenciales).

- Gestión de duplicados por hash y reglas de negocio de primera carga.

Finales Enero (foco integración + madurez funcional)

- Reorganización de navegación y módulos de usuario/admin.

- Fortalecimiento de soporte (tickets, historial, seguimiento).

- Ajustes de UX/validaciones de carga.

- Integración progresiva con GraphQL para operaciones de usuario y carga de Excel.

- Backend GraphQL operativo con PostgreSQL para autenticar, crear usuario y procesar cargas.

3) Evidencia de trabajo enero (fuentes del proyecto)

3.1 Bitácora del proyecto

- La bitácora documenta explícitamente avances de enero 2026 con foco en autenticación, carga masiva, validación y almacenamiento local.

- También refleja avances con correcciones, consolidación funcional y lineamientos de negocio.

3.2 Trazabilidad por commits

- En enero aparecen hitos de integración GraphQL y ajuste funcional, por ejemplo:

  - `Agregar autenticación GraphQL para login`

  - `Integrar CreateUser en carga masiva`

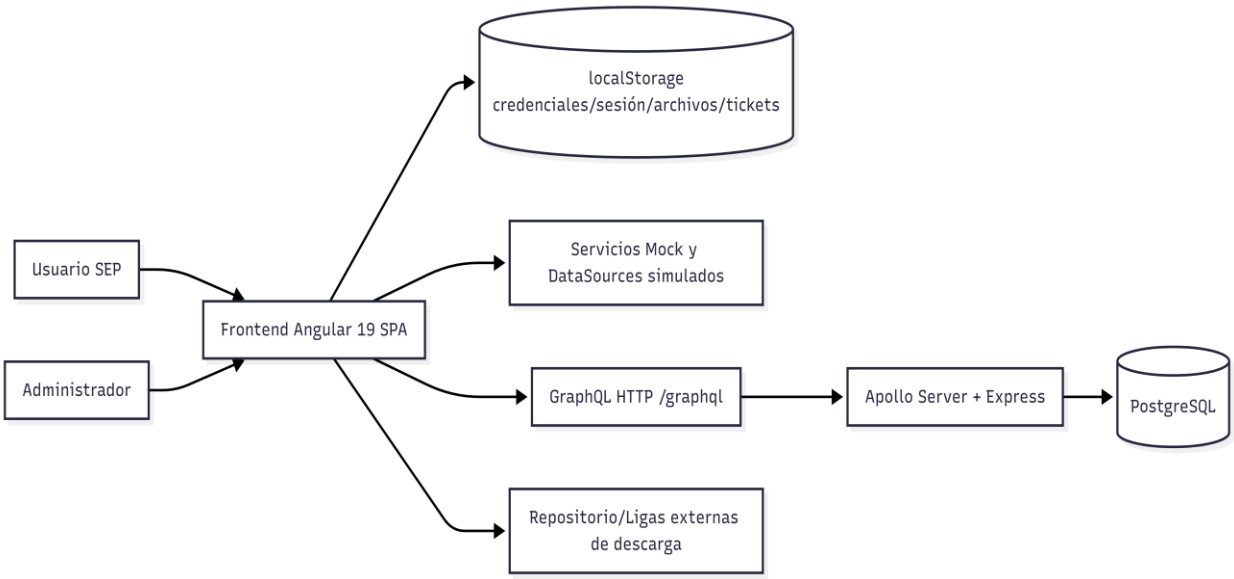
- `Ajustar endpoint GraphQL en frontend`
- `Usar updated_at en autenticacion`

4) Arquitectura lógica y tecnológica del sistema

Este flujo resume cómo conviven dos modos de operación en el periodo reportado:

(a) persistencia local y servicios simulados para continuidad funcional del frontend.

(b) integración progresiva con GraphQL hasta llegar a Apollo + PostgreSQL para operaciones reales.

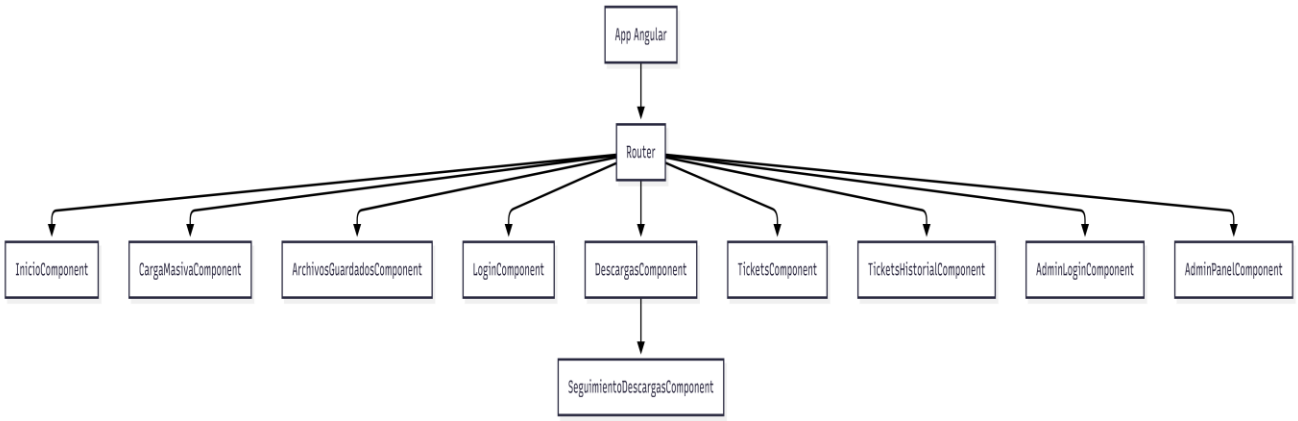


5) Frontend Angular: módulos/páginas implementadas

Rutas funcionales:

- `/inicio`
- `/carga-masiva`
- `/archivos-preescolar`
- `/login`
- `/descargas`
- `/tickets`
- `/tickets-historial`
- `/admin/login`
- `/admin/panel`

Este mapa de componentes muestra el enrutador como eje central de navegación y la distribución de páginas funcionales del sistema, incluyendo flujo de usuario final, soporte y administración.



6) Librerías y frameworks utilizados

6.1 Frontend

- Angular 19 (`@angular/core`, `router`, `forms`, etc.)

- RxJS para flujos reactivos.

- SweetAlert2 para alertas y confirmaciones UX.

- TypeScript 5 + Angular CLI/Karma/Jasmine.

6.2 Backend/API

- Node.js + TypeScript

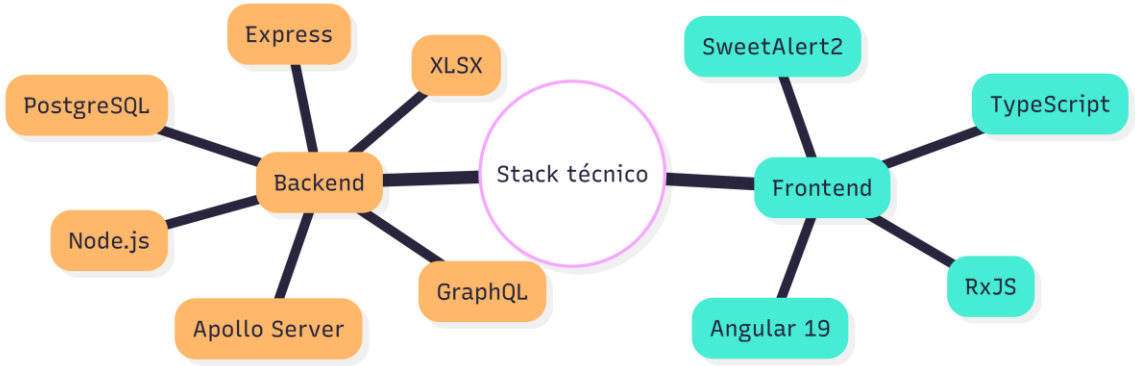
- Apollo Server + GraphQL

- Express, CORS, Helmet, Compression

- PostgreSQL con `pg`

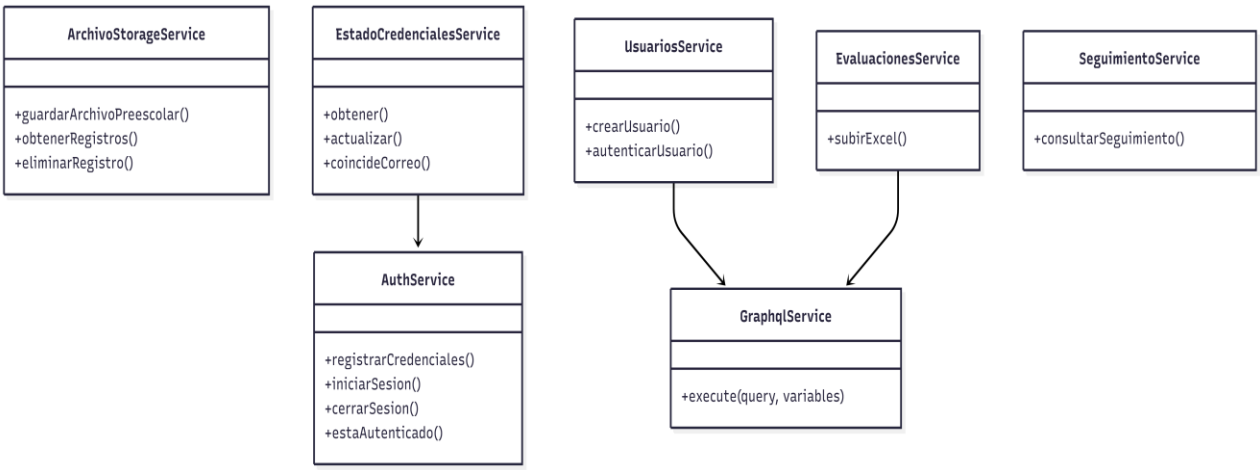
- XLSX para parseo de archivos de carga

El mindmap sintetiza el stack tecnológico realmente usado en el periodo, separando capa frontend y backend para evidenciar frameworks base y librerías complementarias.



7) Arquitectura de servicios frontend

El diagrama de clases representa la separación por responsabilidades en servicios Angular (autenticación, almacenamiento local, estado de credenciales y consumo GraphQL), así como las dependencias entre servicios de dominio y el cliente de API.



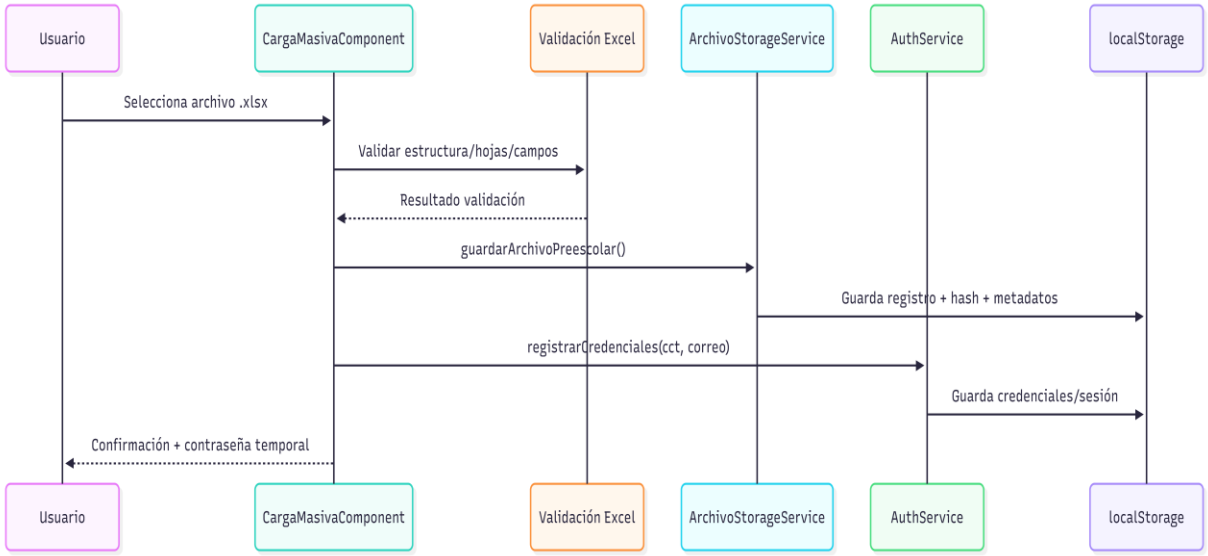
8) Operación basada en localStorage (sin dependencia real de API)

A principios de enero se implementó una arquitectura offline-first/simulada para destrabar entregables de frontend sin bloquearse por backend:

- `AuthService`: controla credenciales/sesión en localStorage.
- `ArchivoStorageService`: guarda archivos y metadatos, controla duplicados por hash.
- `EstadoCredencialesService`: persiste estado de credenciales para continuidad UX.
- `AdminAuthService`: token simulado para panel administrativo.

Servicios de seguimiento/versiones en modo mock.

Esta secuencia describe el flujo típico sin backend obligatorio: validación de archivo, guardado con hash, registro de credenciales y persistencia en localStorage con confirmación al usuario.



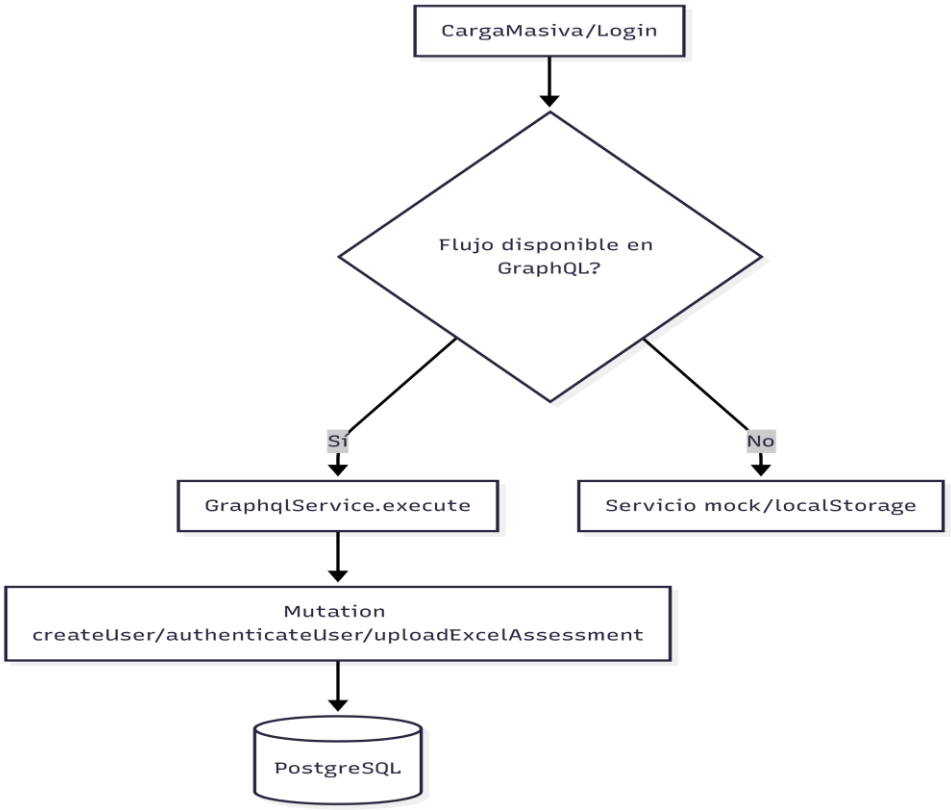
9) Transición a integración GraphQL

A finales de enero se mantiene la experiencia frontend pero se activa integración real por capas para operaciones clave:

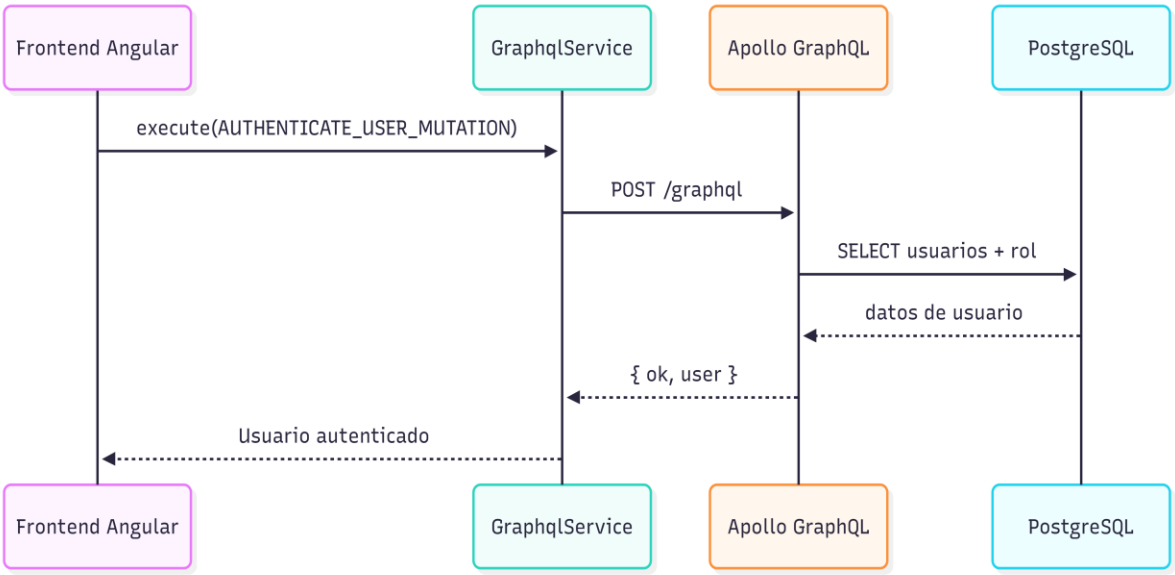
- `GraphqlService` resuelve endpoint (`localhost:4000/graphql` en dev).
- `UsuariosService` consume `createUser` y `authenticateUser`.
- `EvaluacionesService` consume `uploadExcelAssessment`.

Se conserva soporte mock para funcionalidades aún no acopladas totalmente.

Este flujo explica la estrategia híbrida de enero: si el caso de uso ya estaba disponible en GraphQL se enviaba por API; en caso contrario se mantenía servicio mock/localStorage para no frenar operación.

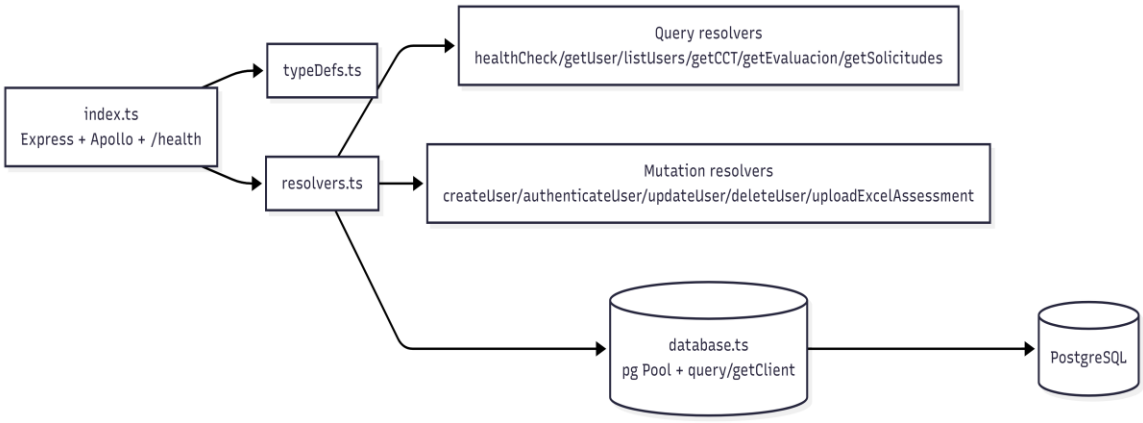


Esta secuencia detalla la autenticación real vía GraphQL: frontend ejecuta mutation, Apollo consulta PostgreSQL, y retorna el objeto de usuario para completar inicio de sesión en la SPA.



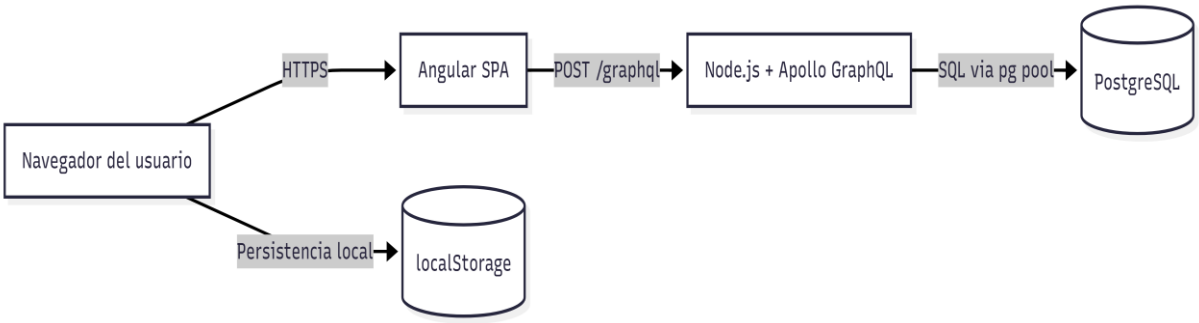
10) Backend GraphQL y componentes

Este componente lógico del backend muestra el punto de entrada (`index.ts`), definición de esquema (`typeDefs`), resolvedores por tipo (queries/mutations) y acceso a datos centralizado mediante `database.ts` hacia PostgreSQL.



11) Diagrama de despliegue técnico

este despliegue resume la topología en ejecución: navegador consumiendo la SPA Angular, comunicación HTTP con endpoint `/graphql`, conexión de API a PostgreSQL y persistencia local complementaria en el cliente.



12) Matriz de componentes

Componente solicitado	Pricipio Enero	Final Enero	Evidencia técnica
Estructura lógica frontend	Implementada	Consolidada	Router + componentes
Módulos Angular/páginas	Implementados	Ajustados UX/navegación	rutas activas
Librerías complementarias	Integradas	Estables	Angular/RxJS/SweetAlert2
Servicios API (frontend)	Simulados	Híbrido mock + real	servicios GraphqlService, UsuariosService, EvaluacionesService
Conexión GraphQL	Parcial/preparación	Operativa en flujos clave	mutations de usuarios/carga
Otros frameworks (backend)	En preparación	Operativos	Express + Apollo + PostgreSQL

13) Conclusión

1. Sí se cumplió el análisis y evolución de arquitectura frontend, con decisiones técnicas alineadas a continuidad operativa (localStorage/mocks) mientras maduraba la capa API.

2. Sí se avanzó en el entregable de diagrama y componentes, al contar con arquitectura de frontend definida, servicios por dominio, integración gradual a GraphQL y backend operativo con PostgreSQL.

3. En enero predominó la simulación controlada (válida para avance funcional) y se materializó la transición a integración real GraphQL en procesos críticos.

José Guadalupe Gutiérrez Arévalo	*David León Gómez*
Jefatura de departamento	*Subdirector de Área*
Joseg.gutierrez@nube.sep.gob.mx	*david.leon@nube.sep.gob.mx*
	55917
ELABORÓ	**REVISÓ**